```
class GPModel(gpytorch.models.ExactGP, botorch.models.gpytorch.GPyTorchModel):
    _num_outputs = 1

    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ConstantMean()
        self.covar_module = gpytorch.kernels.ScaleKernel(
            gpytorch.kernels.MaternKernel( ❶
                nu=2.5, ❶
                ard_num_dims=4 ❶
            ) ❶
        )

    def forward(self, x):
        ...
```

❶ A Matérn 2.5 kernel for four dimensions.

3. Implement a helper function that trains the GP on a given training data set.
   We can simply copy the same helper function `fit_gp_model()` from other notebooks in this chapter, namely `02 - Exercise 1.ipynb`, as we don't need to modify anything in this helper function.
4. Define the settings of our optimization problem.
   We first define the bounds of our search space:

```
lb = 0
ub = 1

bounds = torch.tensor([[lb] * 4, [ub] * 4], dtype=torch.float)
```

   We then specify how many queries can be made, the batch size, as well as the number of experiments to repeat for each policy:

```
num_experiments = 5

num_queries = 100
batch_size = 5
num_iters = num_queries // batch_size
```

5. Run each batch BayesOpt policy we learn in this chapter on the objective function implemented above.
   We first use this code to implement the optimization loop and the outer loop that repeats the experiments for each policy. Specifically, for each individual experiment, we randomly sample one data point inside the search space and then run each BayesOpt policy until we run out of queries. We will see how each policy is defined in the next step.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>